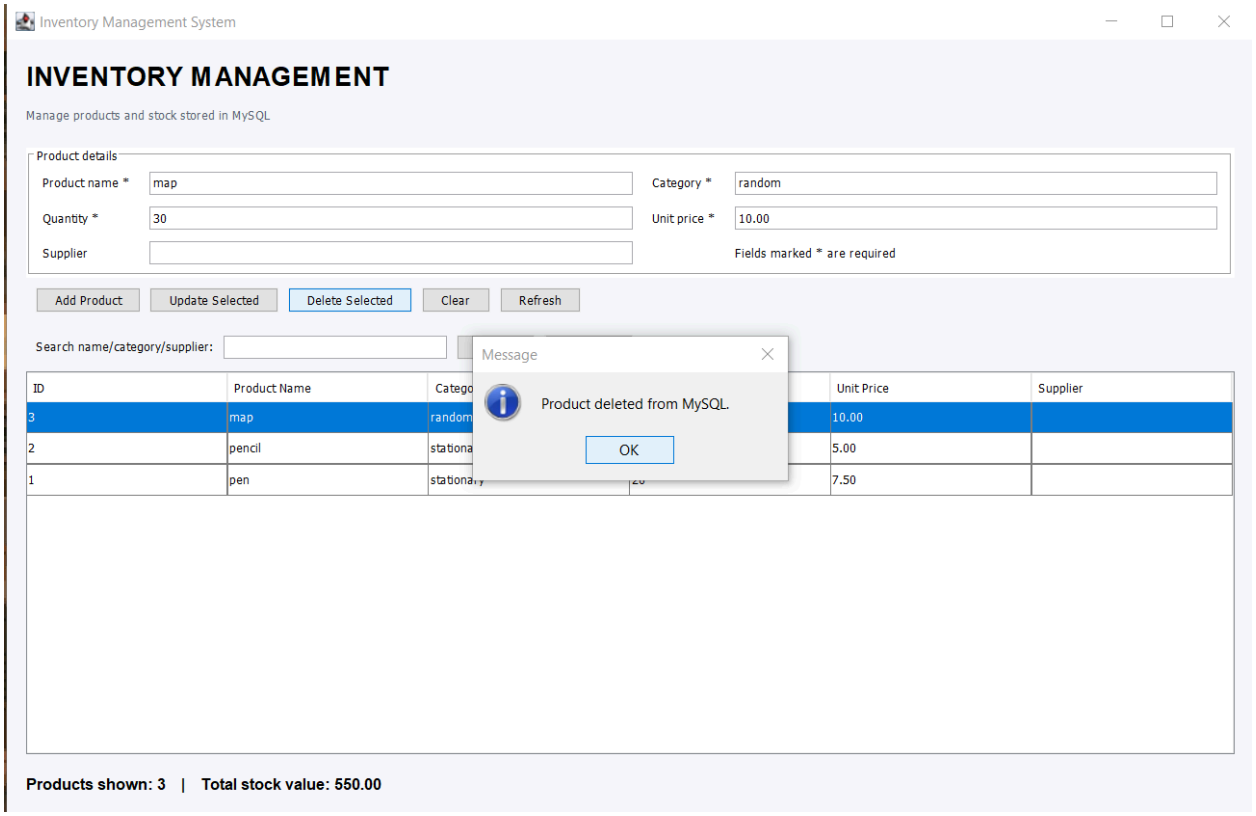
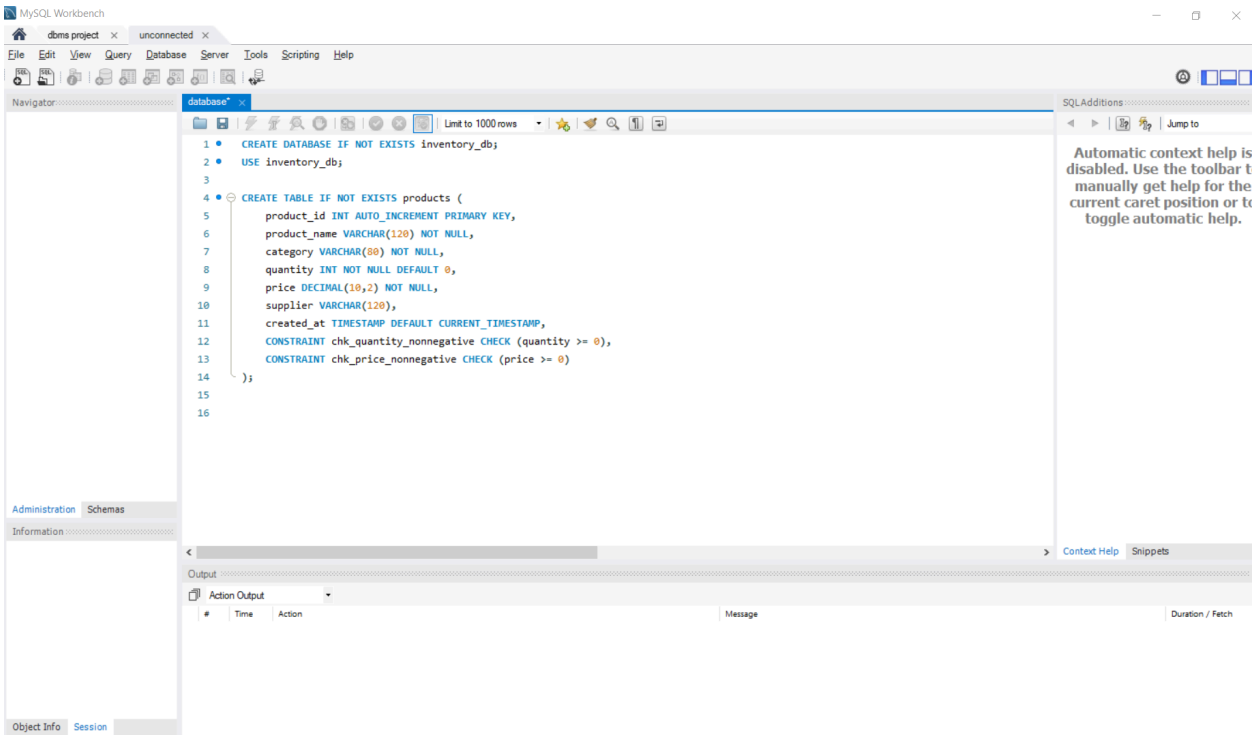
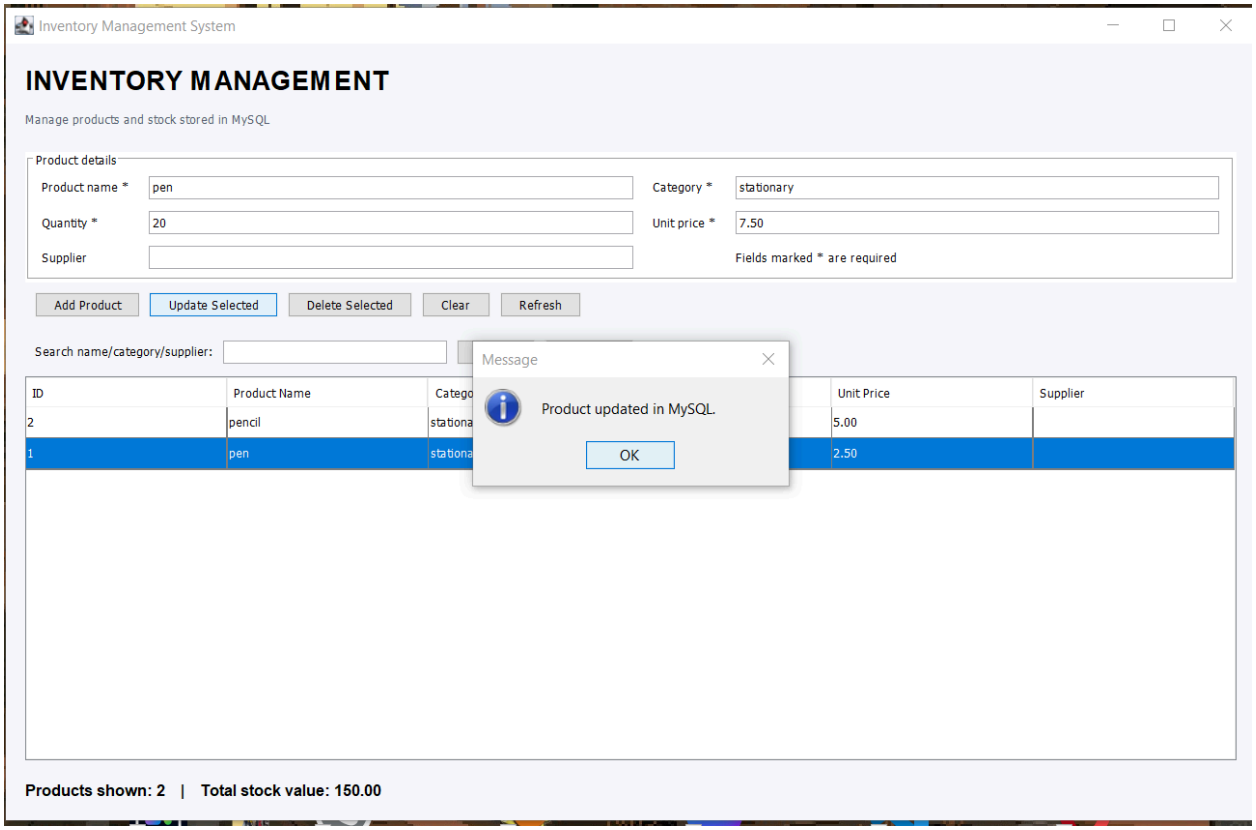
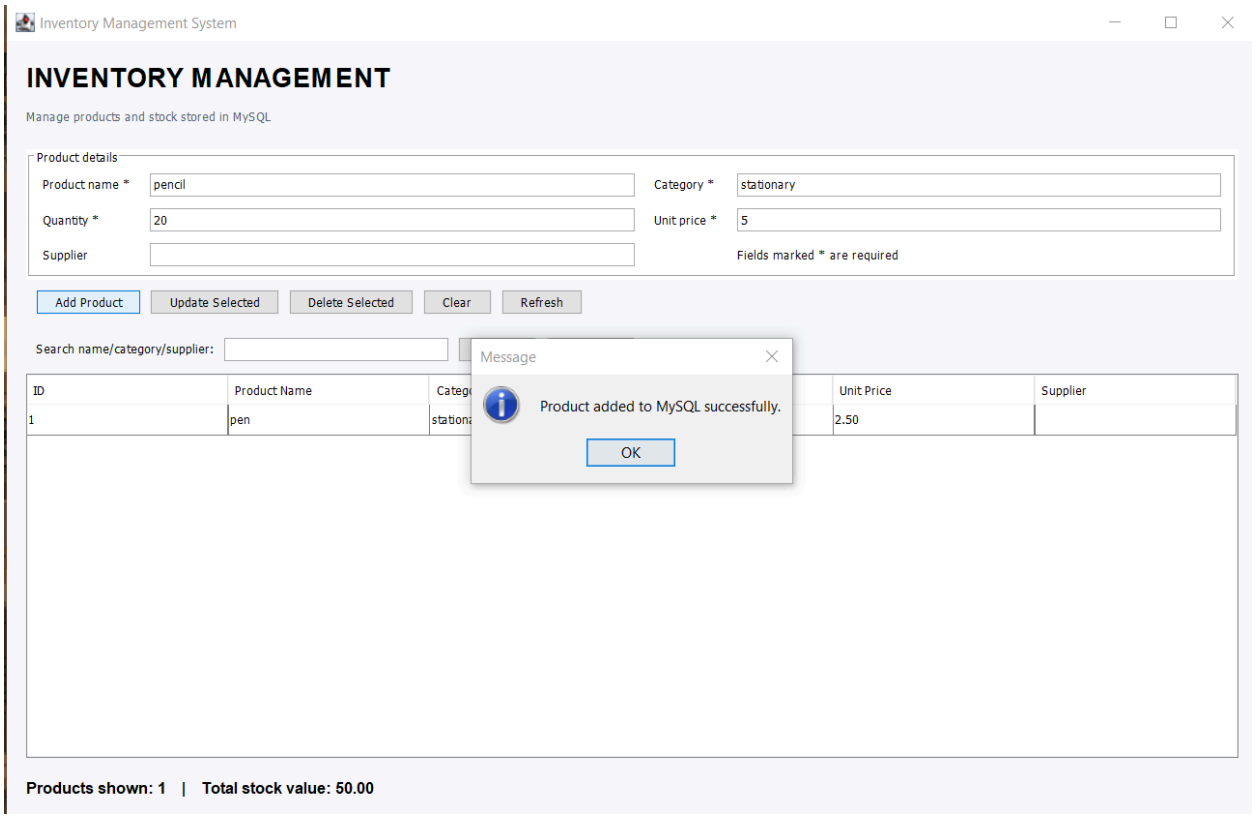

Inventory Management System
INVENTORY MANAGEMENT
Manage products and stock stored in MySQL
Product details
Product name * map
Category * random
Quantity * 30
Unit price * 10.00
Supplier
Fields marked * are required
Add Product
Update Selected
Delete Selected
Clear
Refresh
Search name/category/supplier:
Message
Product deleted from MySQL.
OK
ID
Product Name
Unit Price
Supplier
3
map
10.00
2
pencil
5.00
1
pen
7.50
Products shown: 3 | Total stock value: 550.00


MySQL Workbench
dbms project
unconnected
File Edit View Query Database Server Tools Scripting Help
Navigator
database*
Limit to 1000 rows
CREATE DATABASE IF NOT EXISTS inventory_db;
USE inventory_db;
CREATE TABLE IF NOT EXISTS products (
product_id INT AUTO_INCREMENT PRIMARY KEY,
product_name VARCHAR(120) NOT NULL,
category VARCHAR(80) NOT NULL,
quantity INT NOT NULL DEFAULT 0,
price DECIMAL(10,2) NOT NULL,
supplier VARCHAR(120),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT chk_quantity_nonnegative CHECK (quantity >= 0),
CONSTRAINT chk_price_nonnegative CHECK (price >= 0)
);
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar t manually get help for the current caret position or t toggle automatic help.
Administration Schemas
Information
Object Info Session
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch





package com.inventory;

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.math.BigDecimal;
import java.sql.*;
import java.util.regex.Pattern;

public class InventoryManagementApp extends JFrame {
    private final JTextField nameField = new JTextField(18);
    private final JTextField categoryField = new JTextField(18);
    private final JTextField quantityField = new JTextField(18);
    private final JTextField priceField = new JTextField(18);
    private final JTextField supplierField = new JTextField(18);
    private final JTextField searchField = new JTextField(20);
    private final JLabel summaryLabel = new JLabel("Loading inventory...");
    private final DefaultTableModel model = new DefaultTableModel(
            new Object[]{"ID", "Product Name", "Category", "Quantity", "Unit Price", "Supplier"}, 0) {
        @Override public boolean isCellEditable(int row, int column) { return false; }
    };
    private final JTable table = new JTable(model);
    private Integer selectedProductId = null;

    public InventoryManagementApp() {
        super("Inventory Management System");
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        setMinimumSize(new Dimension(900, 610));
        setSize(1050, 680);
        setLocationRelativeTo(null);
        buildUi();
        refreshProducts();
    }

    private void buildUi() {
        JPanel root = new JPanel(new BorderLayout(12, 12));
        root.setBorder(new EmptyBorder(16, 16, 16, 16));
        root.setBackground(new Color(245, 247, 250));
        setContentPane(root);

        JLabel heading = new JLabel("INVENTORY MANAGEMENT");
        heading.setFont(new Font("SansSerif", Font.BOLD, 22));
        JLabel subheading = new JLabel("Manage products and stock stored in MySQL");
        subheading.setForeground(new Color(90, 100, 115));
```

```
        JPanel titlePanel = new JPanel(new GridLayout(2, 1, 0, 4));
        titlePanel.setOpaque(false);
        titlePanel.add(heading);
        titlePanel.add(subheading);
        root.add(titlePanel, BorderLayout.NORTH);

        JPanel center = new JPanel(new BorderLayout(10, 10));
        center.setOpaque(false);
        root.add(center, BorderLayout.CENTER);

        JPanel form = new JPanel(new GridBagLayout());
        form.setBorder(BorderFactory.createTitledBorder("Product details"));
        form.setBackground(Color.WHITE);
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(5, 8, 5, 8);
        c.fill = GridBagConstraints.HORIZONTAL;
        c.weightx = 1;

        addFormRow(form, c, 0, "Product name *", nameField, "Category *", categoryField);
        addFormRow(form, c, 1, "Quantity *", quantityField, "Unit price *", priceField);
        addFormRow(form, c, 2, "Supplier", supplierField, "", new JLabel("Fields marked * are
required"));

        JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 4));
        actions.setOpaque(false);
        JButton addButton = new JButton("Add Product");
        JButton updateButton = new JButton("Update Selected");
        JButton deleteButton = new JButton("Delete Selected");
        JButton clearButton = new JButton("Clear");
        JButton refreshButton = new JButton("Refresh");
        addButton.addActionListener(e -> addProduct());
        updateButton.addActionListener(e -> updateProduct());
        deleteButton.addActionListener(e -> deleteProduct());
        clearButton.addActionListener(e -> clearForm());
        refreshButton.addActionListener(e -> refreshProducts());
        actions.add(addButton);
        actions.add(updateButton);
        actions.add(deleteButton);
        actions.add(clearButton);
        actions.add(refreshButton);

        JPanel top = new JPanel(new BorderLayout(4, 4));
        top.setOpaque(false);
        top.add(form, BorderLayout.CENTER);
```

```
    top.add(actions, BorderLayout.SOUTH);
    center.add(top, BorderLayout.NORTH);

    table.setRowHeight(26);
    table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
    table.setAutoCreateRowSorter(true);
    table.setFillsViewportHeight(true);
    table.getTableHeader().setReorderingAllowed(false);
    table.getSelectionModel().addListSelectionListener(e -> {
        if (!e.getValueIsAdjusting() && table.getSelectedRow() >= 0) {
            loadSelectedRow();
        }
    });

    JPanel searchPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 4));
    searchPanel.setOpaque(false);
    searchPanel.add(new JLabel("Search name/category/supplier:"));
    searchPanel.add(searchField);
    JButton searchButton = new JButton("Search");
    JButton showAllButton = new JButton("Show All");
    searchButton.addActionListener(e -> searchProducts());
    showAllButton.addActionListener(e -> {
        searchField.setText("");
        refreshProducts();
    });
    searchPanel.add(searchButton);
    searchPanel.add(showAllButton);

    JPanel tablePanel = new JPanel(new BorderLayout(4, 6));
    tablePanel.setOpaque(false);
    tablePanel.add(searchPanel, BorderLayout.NORTH);
    tablePanel.add(new JScrollPane(table), BorderLayout.CENTER);
    center.add(tablePanel, BorderLayout.CENTER);

    summaryLabel.setFont(new Font("SansSerif", Font.BOLD, 13));
    summaryLabel.setBorder(new EmptyBorder(4, 0, 0, 0));
    root.add(summaryLabel, BorderLayout.SOUTH);
}

private void addFormRow(JPanel panel, GridBagConstraints c, int row,
                        String label1, Component field1, String label2, Component field2) {
    c.gridy = row;
    c.gridx = 0; c.weightx = 0;
    panel.add(new JLabel(label1), c);
```

```
        c.gridx = 1; c.weightx = 1;
        panel.add(field1, c);
        c.gridx = 2; c.weightx = 0;
        panel.add(new JLabel(label2), c);
        c.gridx = 3; c.weightx = 1;
        panel.add(field2, c);
    }

    private record ProductInput(String name, String category, int quantity,
                                BigDecimal price, String supplier) {}

    private ProductInput readInput() {
        String name = nameField.getText().trim();
        String category = categoryField.getText().trim();
        String quantityText = quantityField.getText().trim();
        String priceText = priceField.getText().trim();
        String supplier = supplierField.getText().trim();

        if (name.isEmpty() || category.isEmpty() || quantityText.isEmpty() || priceText.isEmpty()) {
            showError("Please fill in product name, category, quantity, and unit price.");
            return null;
        }
        if (name.length() > 120 || category.length() > 80 || supplier.length() > 120) {
            showError("Name, category, or supplier is too long.");
            return null;
        }

        int quantity;
        BigDecimal price;
        try {
            quantity = Integer.parseInt(quantityText);
            if (quantity < 0) throw new NumberFormatException();
        } catch (NumberFormatException ex) {
            showError("Quantity must be a whole number greater than or equal to zero.");
            return null;
        }
        try {
            price = new BigDecimal(priceText);
            if (price.signum() < 0 || price.scale() > 2 || price.precision() > 10) {
                showError("Price must be non-negative, up to 2 decimal places, and fit
DECIMAL(10,2).");
                return null;
            }
        } catch (NumberFormatException ex) {
```

```
            showError("Enter a valid price, e.g. 19.99.");
            return null;
        }
        return new ProductInput(name, category, quantity, price, supplier.isEmpty() ? null :
supplier);
    }

    private void addProduct() {
        ProductInput p = readInput();
        if (p == null) return;
        String sql = "INSERT INTO products (product_name, category, quantity, price, supplier)
VALUES (?, ?, ?, ?, ?)";
        try (Connection conn = Database.getConnection(); PreparedStatement ps =
conn.prepareStatement(sql)) {
            bindProduct(ps, p);
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Product added to MySQL successfully.");
            clearForm();
            refreshProducts();
        } catch (SQLException ex) { databaseError(ex); }
    }

    private void updateProduct() {
        if (selectedProductId == null) {
            showError("Select a product row to update.");
            return;
        }
        ProductInput p = readInput();
        if (p == null) return;
        String sql = "UPDATE products SET product_name=?, category=?, quantity=?, price=?,
supplier=? WHERE product_id=?";
        try (Connection conn = Database.getConnection(); PreparedStatement ps =
conn.prepareStatement(sql)) {
            bindProduct(ps, p);
            ps.setInt(6, selectedProductId);
            int changed = ps.executeUpdate();
            JOptionPane.showMessageDialog(this, changed > 0 ? "Product updated in MySQL." :
"No product was changed.");
            clearForm();
            refreshProducts();
        } catch (SQLException ex) { databaseError(ex); }
    }

    private void deleteProduct() {
```

```
        if (selectedProductId == null) {
            showError("Select a product row to delete.");
            return;
        }
        int answer = JOptionPane.showConfirmDialog(this,
                "Delete product ID " + selectedProductId + "?", "Confirm deletion",
                JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
        if (answer != JOptionPane.YES_OPTION) return;
        try (Connection conn = Database.getConnection();
             PreparedStatement ps = conn.prepareStatement("DELETE FROM products WHERE
product_id=?")) {
            ps.setInt(1, selectedProductId);
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Product deleted from MySQL.");
            clearForm();
            refreshProducts();
        } catch (SQLException ex) { databaseError(ex); }
    }

    private void bindProduct(PreparedStatement ps, ProductInput p) throws SQLException {
        ps.setString(1, p.name());
        ps.setString(2, p.category());
        ps.setInt(3, p.quantity());
        ps.setBigDecimal(4, p.price());
        if (p.supplier() == null) ps.setNull(5, Types.VARCHAR);
        else ps.setString(5, p.supplier());
    }

    private void refreshProducts() {
        loadProducts("SELECT product_id, product_name, category, quantity, price, supplier
FROM products ORDER BY product_id DESC", null);
    }

    private void searchProducts() {
        String term = searchField.getText().trim();
        if (term.isEmpty()) {
            refreshProducts();
            return;
        }
        String sql = """
            SELECT product_id, product_name, category, quantity, price, supplier
            FROM products
            WHERE product_name LIKE ? OR category LIKE ? OR supplier LIKE ?
            ORDER BY product_id DESC
```

```
            """;
        String pattern = "%" + term + "%";
        loadProducts(sql, new String[]{pattern, pattern, pattern});
    }

    private void loadProducts(String sql, String[] params) {
        model.setRowCount(0);
        BigDecimal stockValue = BigDecimal.ZERO;
        int count = 0;
        try (Connection conn = Database.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            if (params != null) {
                for (int i = 0; i < params.length; i++) ps.setString(i + 1, params[i]);
            }
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    int qty = rs.getInt("quantity");
                    BigDecimal price = rs.getBigDecimal("price");
                    stockValue = stockValue.add(price.multiply(BigDecimal.valueOf(qty)));
                    count++;
                    model.addRow(new Object[]{
                            rs.getInt("product_id"),
                            rs.getString("product_name"),
                            rs.getString("category"),
                            qty,
                            price.toPlainString(),
                            rs.getString("supplier") == null ? "" : rs.getString("supplier")
                    });
                }
            }
            summaryLabel.setText("Products shown: " + count + "    |    Total stock value: " +
stockValue.setScale(2, java.math.RoundingMode.HALF_UP));
        } catch (SQLException ex) {
            databaseError(ex);
            summaryLabel.setText("Could not load inventory.");
        }
    }

    private void loadSelectedRow() {
        int viewRow = table.getSelectedRow();
        if (viewRow < 0) return;
        int row = table.convertRowIndexToModel(viewRow);
        selectedProductId = Integer.parseInt(model.getValueAt(row, 0).toString());
        nameField.setText(model.getValueAt(row, 1).toString());
```

```
        categoryField.setText(model.getValueAt(row, 2).toString());
        quantityField.setText(model.getValueAt(row, 3).toString());
        priceField.setText(model.getValueAt(row, 4).toString());
        supplierField.setText(model.getValueAt(row, 5).toString());
    }

    private void clearForm() {
        selectedProductId = null;
        nameField.setText("");
        categoryField.setText("");
        quantityField.setText("");
        priceField.setText("");
        supplierField.setText("");
        table.clearSelection();
    }

    private void showError(String message) {
        JOptionPane.showMessageDialog(this, message, "Input / Selection Error",
JOptionPane.WARNING_MESSAGE);
    }

    private void databaseError(SQLException ex) {
        JOptionPane.showMessageDialog(this,
                "Could not complete the database operation.\n"
                        + "Check MySQL is running and credentials in Database.java are correct.\n\n"
                        + "Details: " + ex.getMessage(),
                "MySQL Error", JOptionPane.ERROR_MESSAGE);
    }

    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) { }
        SwingUtilities.invokeLater(() -> {
            try {
                Database.initialize();
                new InventoryManagementApp().setVisible(true);
            } catch (SQLException ex) {
                JOptionPane.showMessageDialog(null,
                        "Unable to connect to MySQL or initialize inventory_db.\n\n"
                                + "1. Start the MySQL server.\n"
                                + "2. Edit Database.java and set your MySQL password.\n"
                                + "3. Ensure the MySQL user can create a database/table.\n\n"
                                + "Details: " + ex.getMessage(),
```

```
                    "Startup Error", JOptionPane.ERROR_MESSAGE);
            }
        });
    }
}
```